Deepfake Detection

By :Sparsh rastogi

2301AI56

01 Introduction
02 System Architecture
03 Dataset Exploration
04 Pre-processing
05 Model Architecture
06 Training Workflow
07 Prediction Workflow
08 Tools and Technologies
09 Results
10 Demo

❑ Introduction

- Deep fake is a technique for human image synthesis based on artificial intelligence.
- Deep fakes are created by combing and superimposing existing images and videos onto source images or videos using a deep learning technique known as generative adversarial network.

- **Can we detect Deep fakes with naked eyes?**

Real or Fake?

Real

Fake

Fake

Fake

Fake

Real

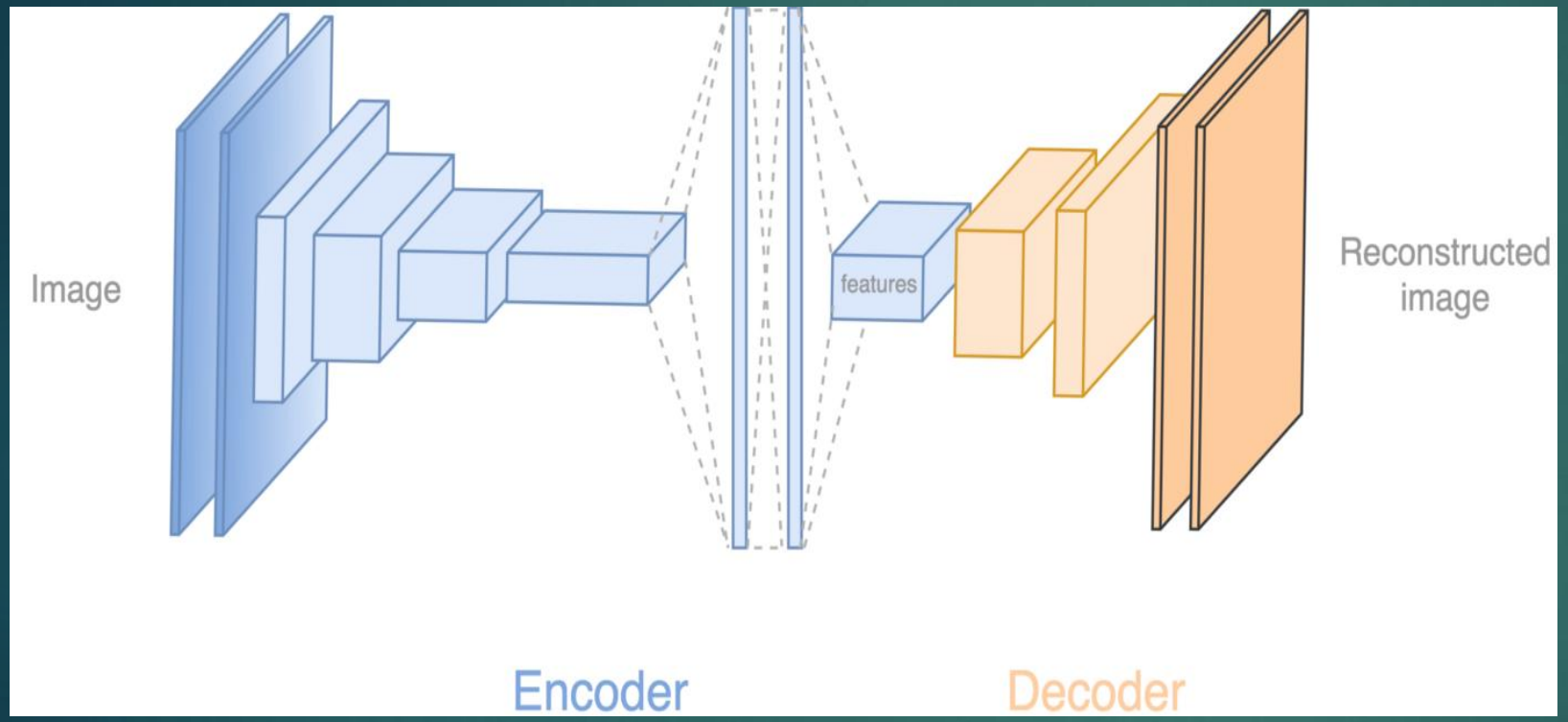
[Rössler et al. 2019]

Source

• Why Deep Fake Detection ?

- Fake News
- Malicious hoaxes
- Financial fraud
- Celebrity unusual video
- Revenge porn
- Politician videos

❑ How Deep Fakes Are Created ?



Tools for deep fake creation.

- Faceswap
- Faceit
- DeepFaceLab
- DeepfakeCapsuleGAN
- Large resolution facemasked

PART 1

Dataset Exploration & Benchmark Selection

Surveying the deepfake video detection landscape

FaceForensics++

2019

Videos

~7,000

Source

YouTube

Landmark academic benchmark for face-swap detection

140K - REAL & FAKE

2020

Images

~140,000

Source

Flickr,Nvidia

Benchmark, higher visual quality deepfakes

DFDC

2020

Videos

~124,000

Source

Industry-backed

Largest public face-swap dataset; industry standard

Sources: FaceForensics++ (Rössler et al., 2019) · Celeb-DF (Li et al., 2020) · DFDC (Dolhansky et al., 2020)

Why DFDC is the Industry Standard

Unmatched Scale

Largest publicly available face-swap video dataset — 124k total labeled videos.

Real-World Complexity

Diverse lighting, varied backgrounds, heavy compression to mimic 'in the wild' content.

Actor Diversity

3,400+ consented paid actors spanning diverse genders, ages, and ethnicities.

Multiple Threat Vectors

8 manipulation techniques (GANs, voice swaps) forcing model generalization.

BACKED BY INDUSTRY LEADERS

- Amazon Web Services
- Meta (Facebook)
- Microsoft
- Partnership on AI
- Leading Academics

DATASET ANATOMY

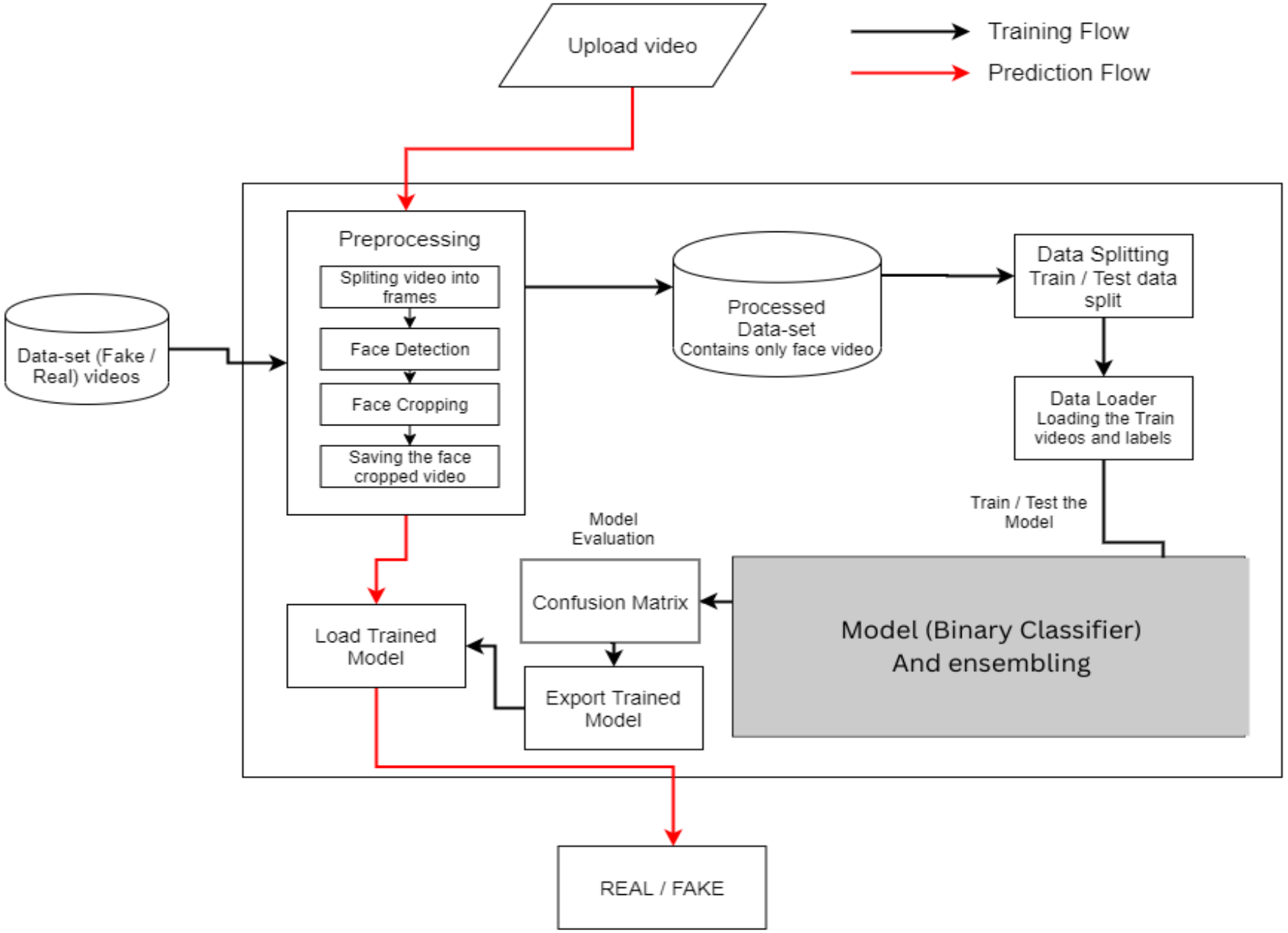
~124k

TOTAL LABELED VIDEOS

100,000 Fake | 19,154 Real

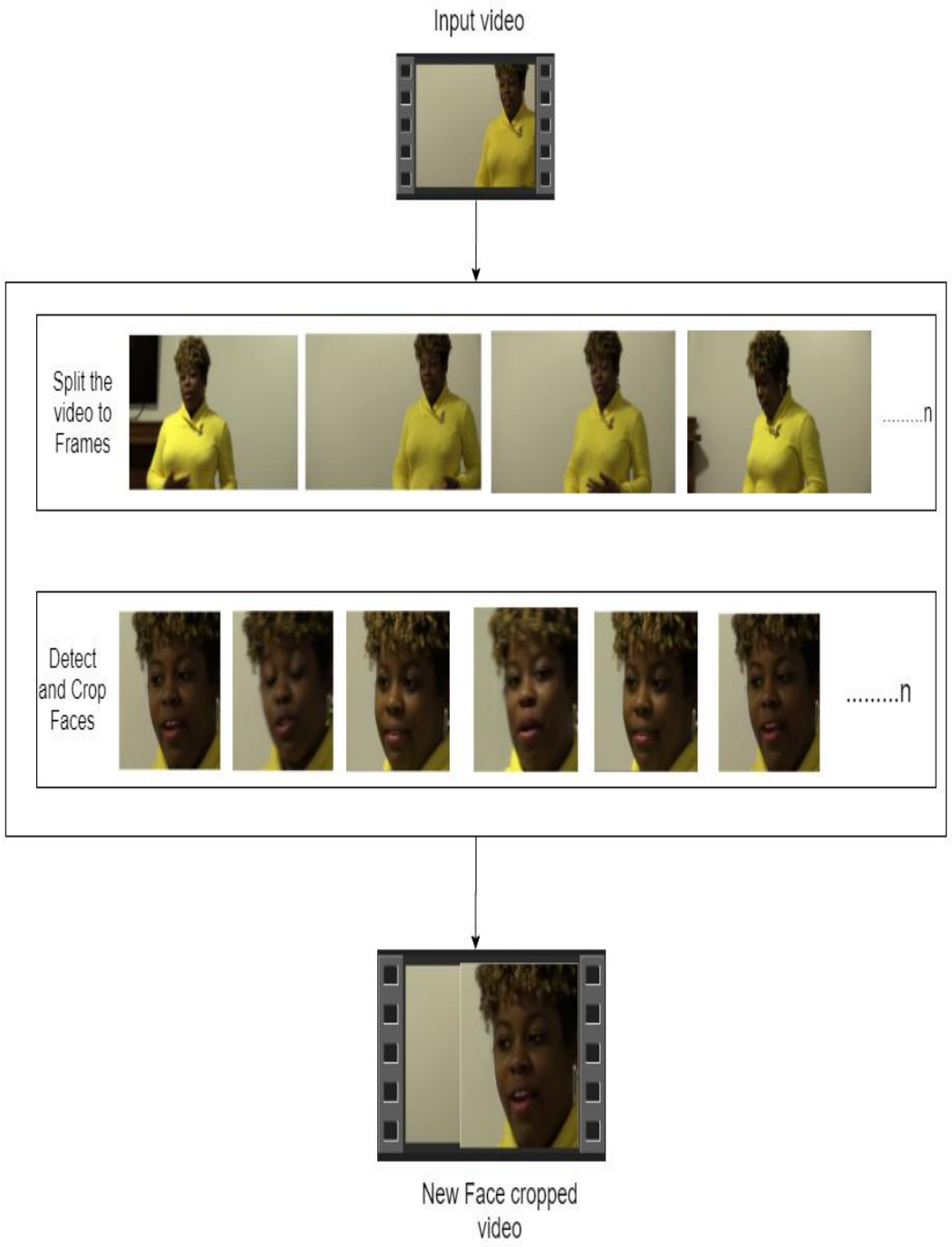
+ 5,000 test videos

System Architecture (OVERALL)



Pre-processing

1. SPLIT VIDEO INTO FRAMES
2. FACE DETECTION
3. CROPING & ADDING MARGINS
4. AUGMENTATION
5. SAVING THE FACE CROPPED FRAMES



PART 3

Data Preprocessing & Augmentation Pipeline

① Frame Sampling

Dataset 1

Base Models

1 frame

Single random frame per video

Dataset 2

Meta / Stacked

10 frames

10 random frames per video

② Face Extraction

Engine

BlazeFace

@humananalog kernel (offline)

Output

Uniform resize

256×256

All faces standardized

③ Augmentation Strategy

Primary

p = 0.50

JPEG + Downscale Blur & Gaussian noise

Secondary

Brightness + Flip

Horizontal flipping

④ Class Imbalance

Single Model

Training

Oversampling

ImbalancedDatasetSampler

Stacked Model

Training

Undersampling

Equal class representation

⚠ **Tested & Discarded:** · Cutout · HueSaturation · 90° Rotation

BlazeFace Face Extractor

@humananalog kernel — offline face detection pipeline

Isotropic Resizing

Maintains aspect ratio via zero-padding on the shorter side, ensuring uniform 256×256 output without distortion.

256×256

output px

Anchor Boxes

Uses a fixed set of precomputed anchors (anchors.npy) that must be loaded — raw coordinates are meaningless without them.

anchors .npy

required file

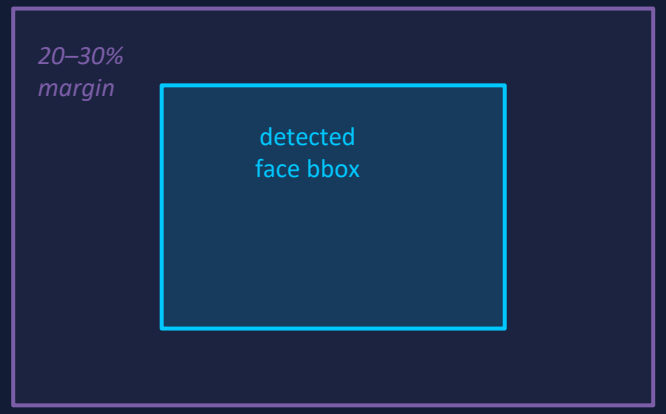
Performance

Processes ~150–200 frames per second on a T4 GPU — critical for handling DFDC's massive 124k video test set at scale.

200 fps

T4 GPU

BOUNDING BOX STRATEGY



Tight crop ✗

Padded crop ✓

Adding a 20–30% margin around the detected bounding box captures contextual pixels (hair, jaw, ears) that deep fake artifacts often appear at the face periphery.

150–200	fps on T4 GPU	**256×256**	uniform output	**20–30%**	face margin	**Offline**	batch processing

(Discarded) Model Architecture

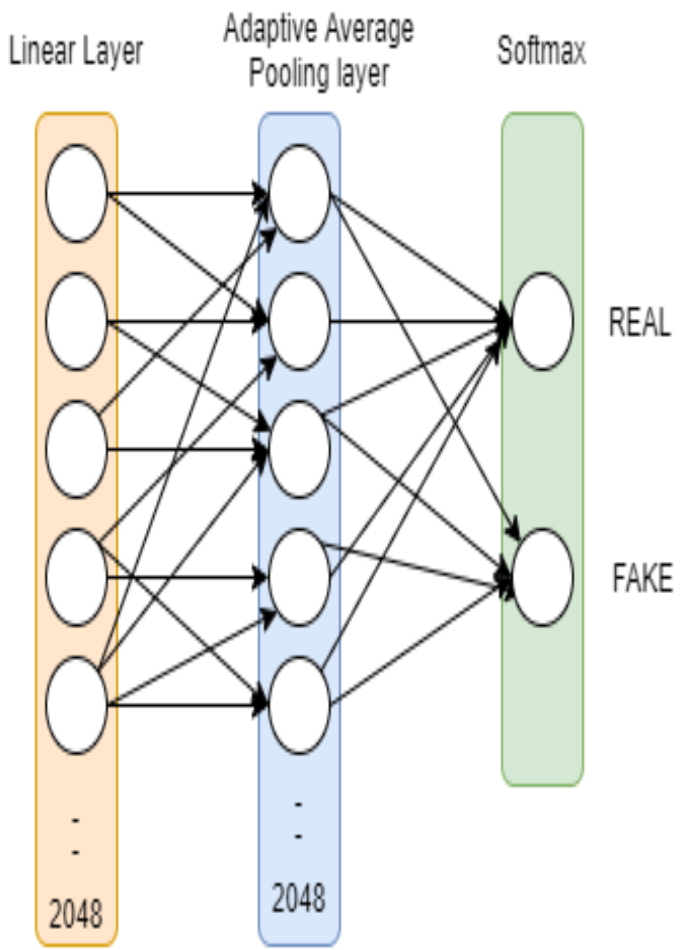
ResNext-50

To do feature extraction
With details

Sequential Layer

1 LSTM layer and ReLU Activation function

To deal with time dependent video data and analyze the relation between frames



PART 4

Why Not ResNet-50 + LSTM?

The spatiotemporal baseline — and why it failed on DFDC

ResNet-50

Extracts spatial features frame-by-frame — blending artifacts, pixel anomalies at face boundaries.

↓

LSTM

Analyzes temporal sequences — flickering, unnatural movement across consecutive frames.

↓

✗ Poor validation on DFDC

THE PROBLEMS

Underfitting

RNNs struggled to learn generalized deepfake patterns on DFDC. Validation scores lagged behind 2D CNNs consistently.

CPU / Memory Bottleneck

Sequential video chunks through heavy CNN + RNN stacks caused massive slowdowns — unacceptable at DFDC's scale.

Nature of the Artifacts

DFDC fakes primarily show single-frame spatial glitches (blending edges, warped pixels) — not complex temporal errors.

THE PIVOT

✗ Sequential Video Analysis

↓

✓ Bag of Independent Images

Replace LSTM with Stacking Ensemble (Phase 2 MetaModel)

EfficientNet Architecture

Compound Scaling for Convolutional Neural Networks • Tan & Le, 2019

NETWORK PIPELINE

MBConv Block (Inverted Residual + Squeeze-Excitation)

COMPOUND SCALING

Depth (d)

More layers

Width (w)

More channels

Resolution (r)

Larger input

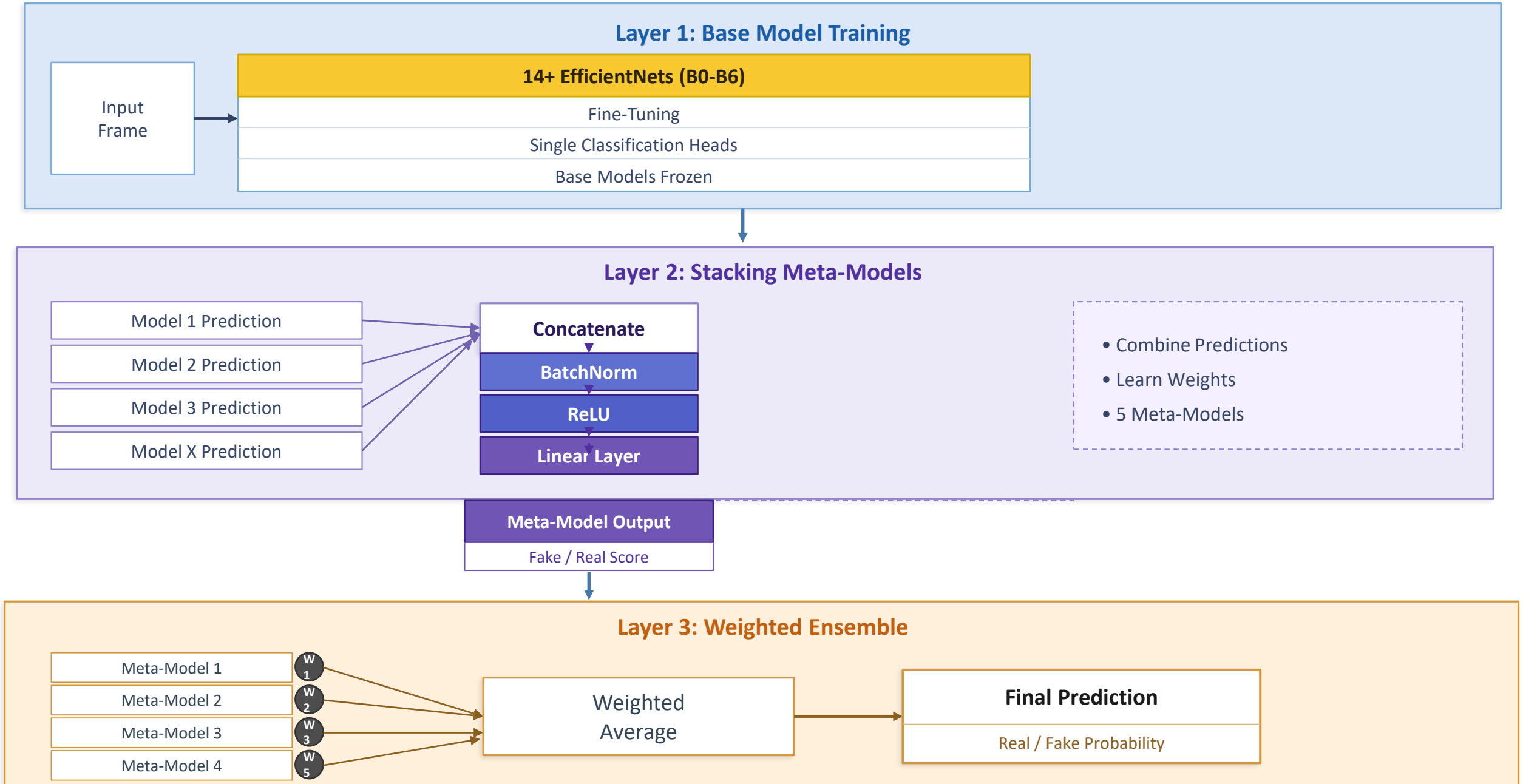
ϕ (phi) coefficient

B0–B7	5.3M	84.4%	8.4×
Model Variants	B0 Parameters	B7 Top-1 (ImageNet)	Smaller than GPipe

Model Scale: B0 B1 B2 B3 B4 B5 B6 B7

Each Bx variant uniformly scales Depth · Width · Resolution by compound coefficient ϕ

Model Architecture & Ensembling Strategy



Experimental Results

DeepFake Detection Challenge (DFDC) • EfficientNet Backbone Variants + Meta-Ensemble

RAW MODELS (14 variants)

Model	Train	Accuracy	Log Loss
efficientnet-b2	T1	0.8548	0.3377
efficientnet-b3	T1	0.8574	0.3456
efficientnet-b4	T1	0.8579	0.3384
efficientnet-b6	T1	0.8603	0.3319
efficientnet-b0	T2	0.8617	0.3698
efficientnet-b1	T2	0.8411	0.3606
efficientnet-b2	T2	0.8660	0.3560
efficientnet-b3	T2	0.8486	0.3612
efficientnet-b3	T1v2	0.8636	0.3283
efficientnet-b6	T1v2	0.8594	0.3229
tf_b1_ns	NS	0.8571	0.3341
tf_b3_ns ★	NS	0.8712	0.3277
tf_b4_ns	NS	0.8709	0.3153
tf_b6_ns ★	NS	0.8733	0.3157

★ Best raw: tf_efficientnet_b6_ns — Acc 0.8733 | Loss 0.3157

META MODELS (5 ensembles)

Input Slice	# Models	Log Loss
slice(4, 8)	4	0.3064
slice(0, 4)	4	0.2919
slice(0, 8)	8	0.3028
slice(0, 10)	10	0.2630
slice(0, 14) ★	14	0.2648

★ Best meta: slice(0,10) — Loss 0.2630

LOG LOSS IMPROVEMENT

FINAL ENSEMBLE RESULT

14 raw + 5 meta-models | Ensemble log loss: 0.2630 |

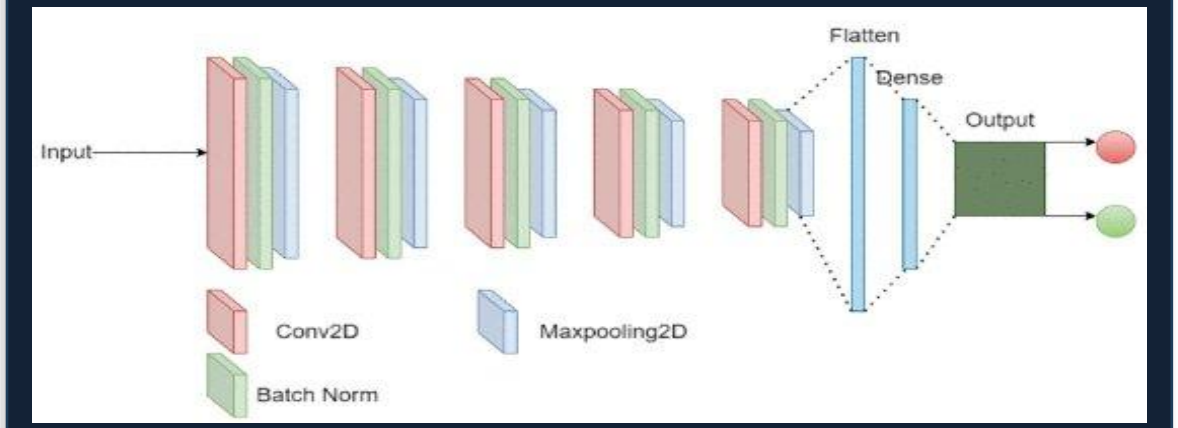
RANK 14 on DFDC Leaderboard

Custom Architectures (Channel-wise Attention)

CNN Module & SE Block Attention

Sequential convolutional feature extraction with channel-wise recalibration

A. CNN MODULE



5× Conv2D Layers
Filters: 32 → 64 → 128 → 256 → 512

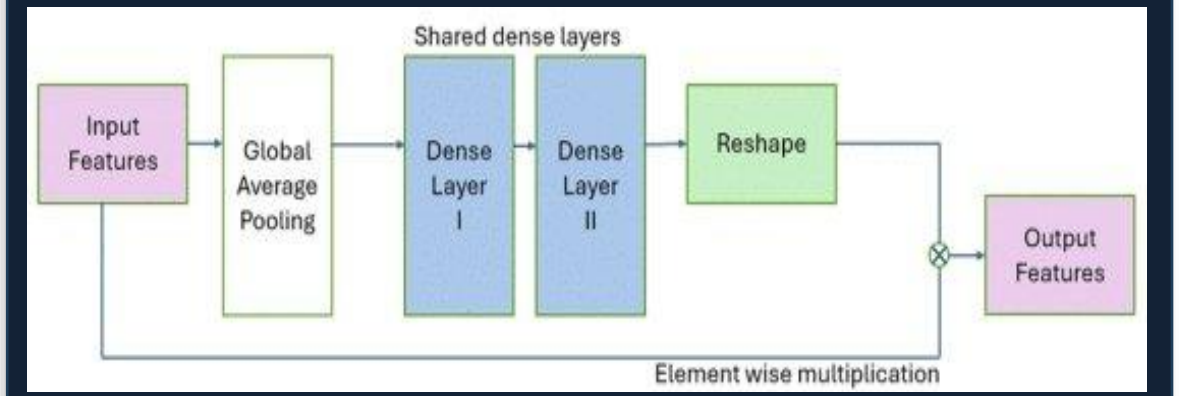
Kernel & Activation
3×3 kernel | ReLU | He_Normal init

MaxPooling2D
Pool size (2,2) — downsampling

Batch Normalization
Stabilize & accelerate training

Flatten → Dense → Output
Adam lr=0.001 | Categorical Cross-Entropy

B. SE BLOCK ATTENTION



SQUEEZE

$z_c = (1/H \times W) \Sigma u_c(i,j)$

Global Average Pooling compresses each channel into a single scalar.

EXCITATION

$w = ReLU(W_1 z + b_1) \rightarrow s = \sigma(W_2 w + b_2)$

Two dense layers (ReLU → Sigmoid) learn channel-wise importance weights.

SCALING

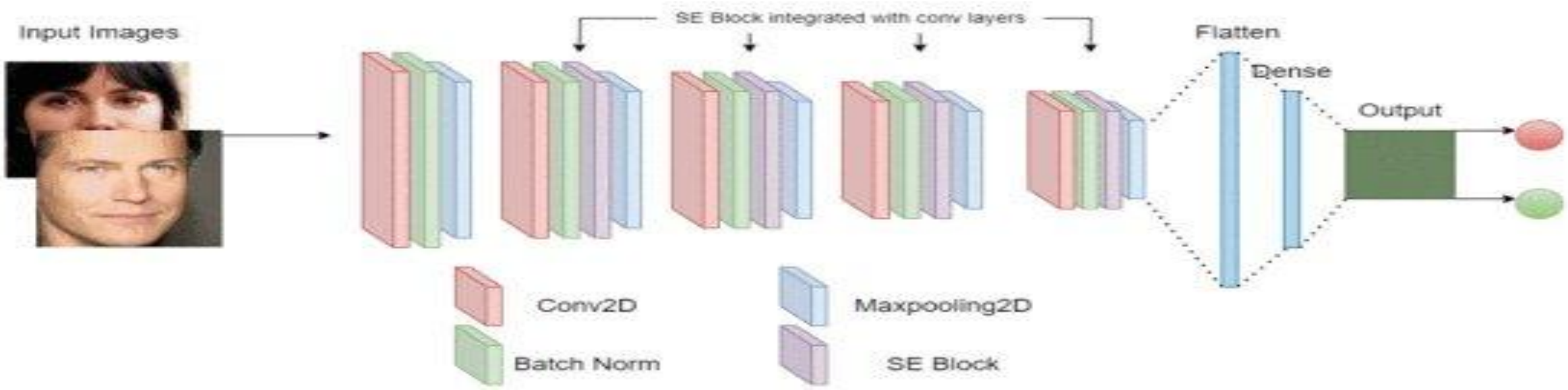
$v_c(i,j) = s_c \cdot u_c(i,j)$

Element-wise multiplication recalibrates feature maps by learned weights.

SE blocks strengthen shared representations in early layers; become class-specific in deeper layers (Hu et al., 2018)

Proposed Architecture: Sequential CNN + SE Attention

SE blocks are integrated after Batch Normalization in deeper conv layers — output feeds MaxPooling before next stage



FULL PIPELINE FLOW

Input Images → Preprocess & Augment → Conv2D + BN → SE Block (×4) → MaxPool 2D → Repeat 4× → Conv2D (final) → Flatten → Dense → Softmax Output

KEY DESIGN DECISIONS

SE After BN

SE block placed after Batch Norm, output flows to MaxPool before next conv

Deeper Integration

SE applied to 4 of 5 conv layers — early: shared features; later: class-specific

Skin Texture Focus

Channel recalibration forces model to weight skin texture features for deepfake detection

Minimal Overhead

SE adds marginal compute vs. building a new architecture from scratch

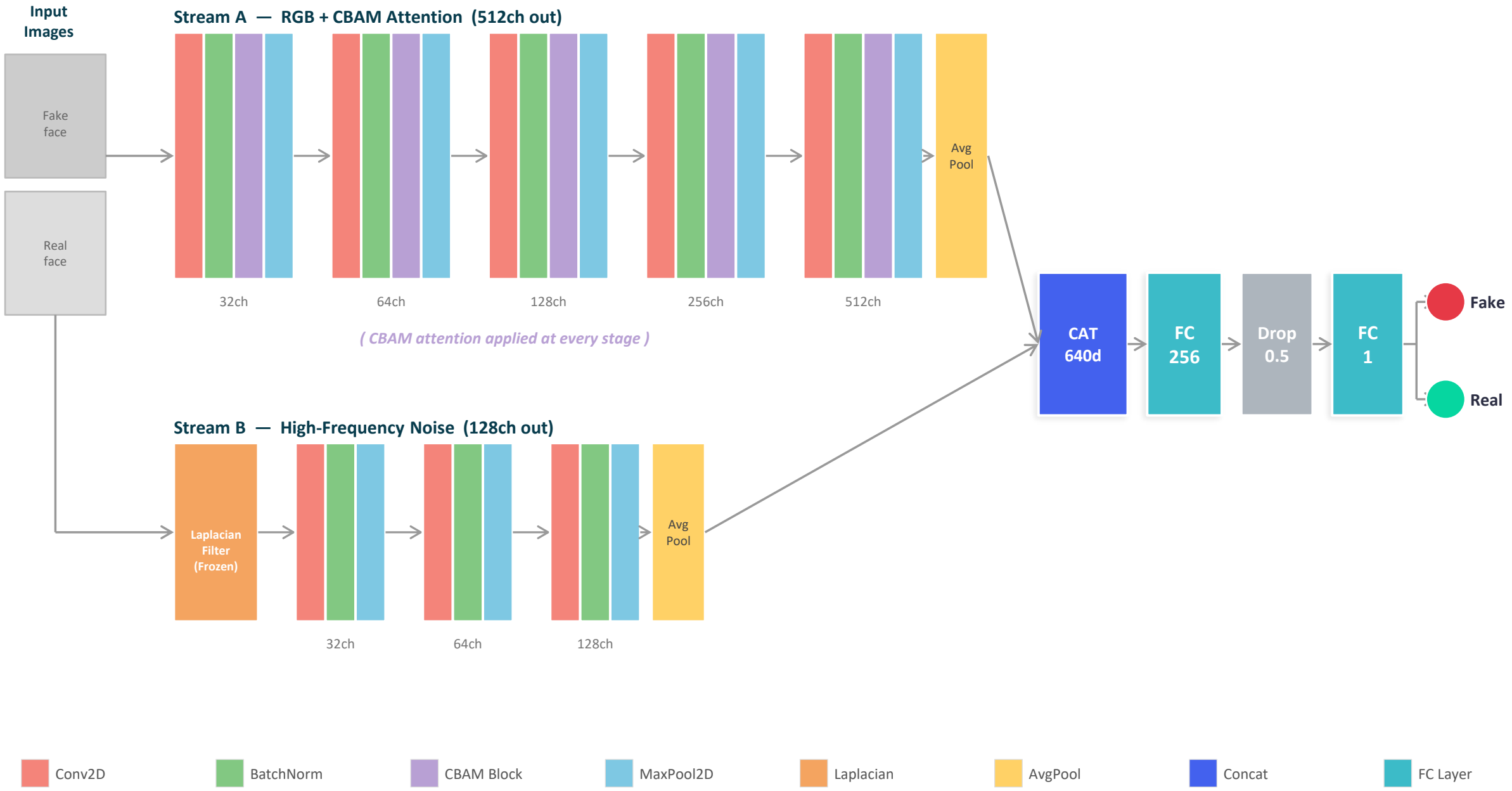
Fig. 3 — Overview of proposed Sequential CNN + SE Block model for fake face detection

Part 1

Dual-Stream CBAM CNN

Late Fusion · RGB + Laplacian Noise · CBAM Attention · 99.47% Accuracy

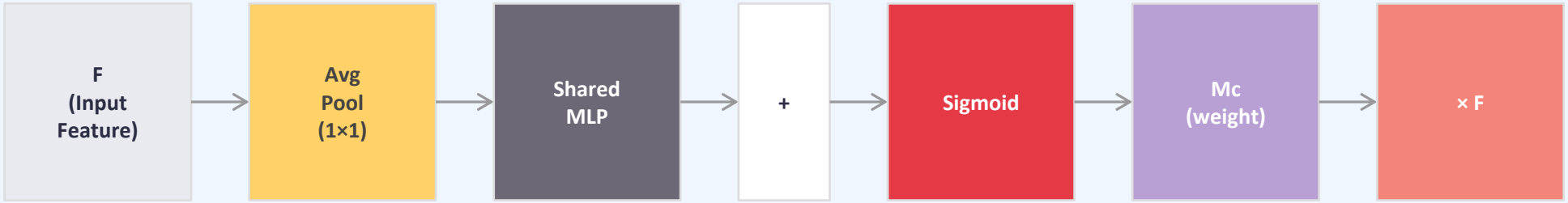
Dual-Stream CBAM CNN — Architecture Diagram



CBAM Block — Convolutional Block Attention Module

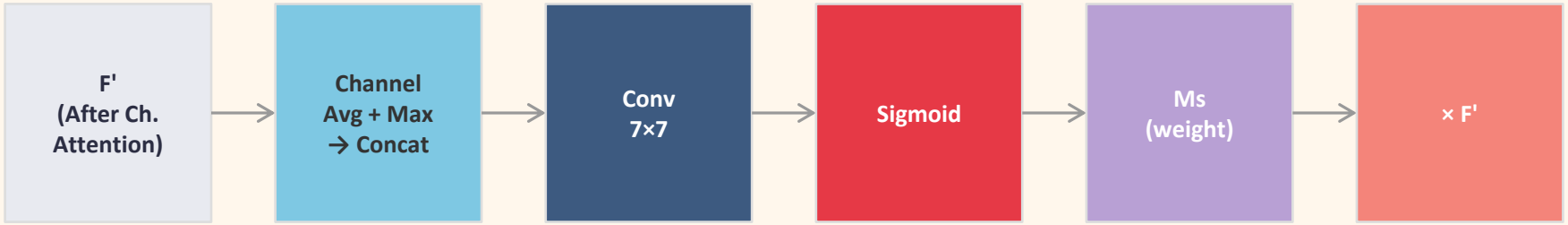
Applied after every Conv stage in Stream A to sharpen spatial + channel focus on artifact regions

① Channel Attention



(MaxPool path also feeds Shared MLP — outputs are summed)

② Spatial Attention



Channel Attention → Spatial Attention applied in sequence

Output = refined feature map with suppressed irrelevant regions.

Final Test Results — Dual-Stream CBAM CNN

99.47%

Overall Accuracy

Loss: 0.0157

Class	Precision	Recall	F1-Score	Support
Fake	0.9984	0.9910	0.9947	10,000
Real	0.9911	0.9984	0.9947	10,000
Macro Avg	**0.9947**	**0.9947**	**0.9947**	**20,000**

Key Takeaways

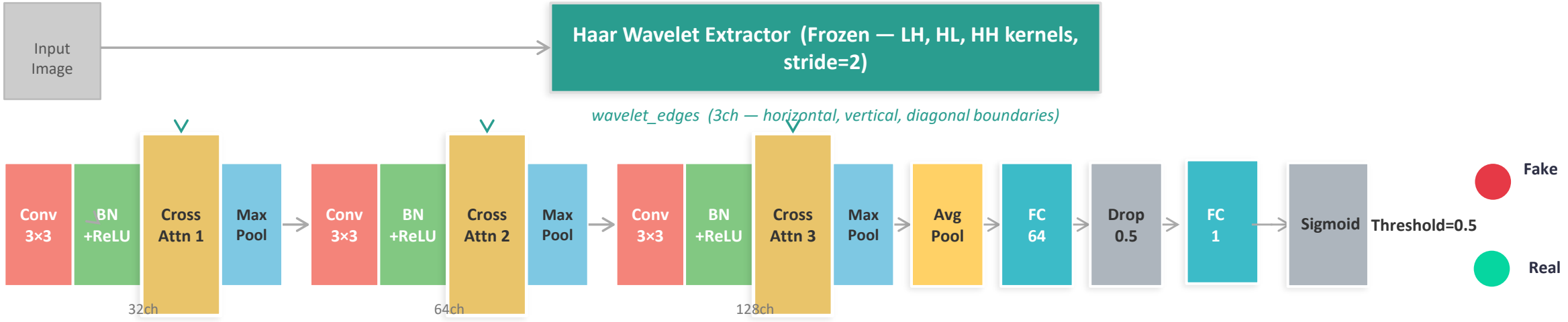
1. Near-perfect precision & recall across both classes — no class bias
2. Separating RGB features from raw Laplacian noise before the classifier is highly effective
3. Model trained on 140k Real & Fake Faces dataset (StyleGAN-based, aligned faces)
4. Cross-dataset evaluation on DFDC / FaceForensics++ is the next challenge

Part 2

WGC-Net

Wavelet-Guided Cross-Attention Network · Early Fusion · 97.45% Accuracy

WGC-Net — Architecture Diagram



Key Insight: Wavelet boundaries injected at 3 depths — spatial convolutions attend to mathematical blending edges at every scale. Stronger inductive bias than late fusion.

Conv2D | BN + ReLU | Wavelet Cross-Attn | MaxPool2D | Haar Extractor | AvgPool | FC Layer

Wavelet Cross-Attention — The Novel Injection Mechanism

Applied 3× inside WGC-Net — guides spatial convolutions with mathematical wavelet truth at each depth

① Haar Wavelet Extractor

LH Kernel (Horizontal Edges)

−1	−1
+1	+1

÷ 2

HL Kernel (Vertical Edges)

−1	+1
−1	+1

÷ 2

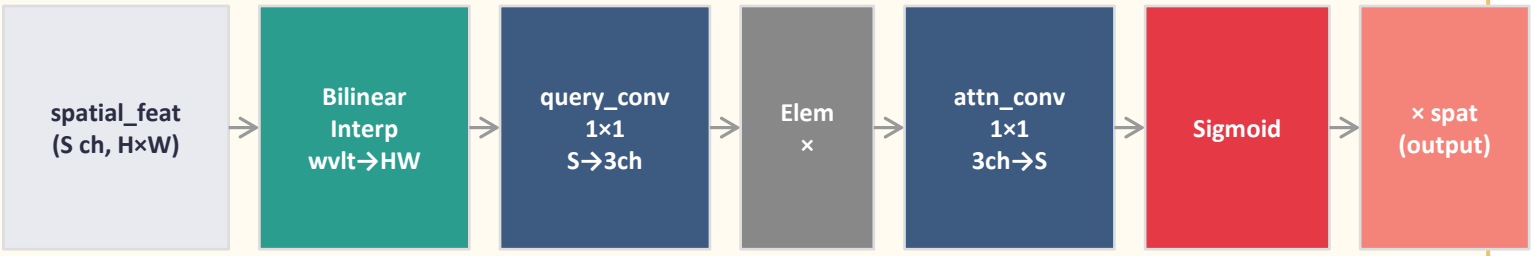
HH Kernel (Diagonal Edges)

+1	−1
−1	+1

÷ 2

Stride = 2 → Downsamples by 2×

② WaveletCrossAttention — Forward Pass




```
# 1. Resize wavelet map to match spatial feature dimensions
wavelet_resized = interpolate(wavelet_edges, size=spatial_feat.shape[2:])

# 2. Compress spatial channels → query in wavelet space
query = query_conv(spatial_feat)           # [B, S, H, W] → [B, 3, H, W]

# 3. Which spatial regions match wavelet boundaries?
attention_map = query * wavelet_resized

# 4. Expand back and gate the spatial features
attention_weights = sigmoid(attn_conv(attention_map))  # [B, S, H, W]
return spatial_feat * attention_weights    # Refined features
```

Project Success & Benchmark Achievements

Slide 1 · Executive Summary & Performance

🎯 The Goal Achieved

Successfully engineered two custom deepfake detection architectures integrating explicit mathematical priors, rivalling the accuracy of massive industry-standard models.

💡 Key Takeaway

It is not strictly necessary to rely on black-box spatial feature extractors. By explicitly feeding high-frequency edge data into the network, we achieve near-perfect classification on high-quality deepfakes.

Final Performance — 140k Dataset

EfficientNet-B0 *(Baseline)*	98.19%
Dual-Stream CBAM *(Custom)*	99.47%
WGC-Net *(Custom)*	97.45%

Custom model within 0.17% of baseline

99.47%
Best Custom Accuracy

97.45%
WGC-Net Accuracy

0.9947
Macro F1-Score

Sparsh Rastogi | Roll No. 2301AI56

Insights from the Dual-Stream CBAM Architecture

Slide 2 · Architectural Findings — Late Fusion

99.47%
Accuracy

🎯 The Strategy

Late Fusion — RGB and high-frequency noise processed completely separately, combined only at the final classification head. Streams never contaminate each other during feature learning.

⚙️ The Mechanism

A frozen 3×3 Laplacian filter acts as a dedicated Noise Stream — a pure mathematical extractor whose weights never change. A hard inductive prior the network cannot bypass.

🔬 The Finding

Isolating frequency data from spatial data until the final FC layer enables highly independent feature representations. CBAM forces the RGB stream to focus on spatial anomalies, perfectly complementing the Laplacian noise.

📊 The Result

Highest custom accuracy: 99.47% · Loss 0.0157 · F1 0.9947. Near-perfect symmetric precision/recall across both classes — zero class bias.

Accuracy: **99.47%** Loss: **0.0157** F1 (Fake & Real): **0.9947**

🔥 Grad-CAM Attention Visualization — CBAM Stream A

Original Image ***Grad-CAM Overlay***

Instead of being distracted by the background, hair, or clothing, the network's attention is intensely focused on the specific facial regions where blending, warping, or generation artifacts occur.

The Vulnerability of Frequency-Based Detection

Slide 4 · The Generalization Gap — Cross-Dataset Evaluation

⚠️ Cross-Dataset Stress Test Results

Model	In-dist.	Out-dist.	Fake F1
EfficientNet-B0	98.19%	~58%	0.16
Dual-Stream CBAM	99.47%	~59%	0.07
WGC-Net	97.45%	~58%	Collapse

↓ Performance drop: ~41% on out-of-distribution data

Why This Happens:

Deepfake detection is heavily reliant on how the fakes are generated and processed. High-quality GANs leave sharp, high-frequency artifacts (which our models excelled at finding). However, social media video compression acts as a low-pass filter, erasing those mathematical boundaries.

🔬 Scientific Findings & Implications

🎯 High-quality GANs leave sharp, high-frequency artifacts in raw images — which our models excelled at detecting (99%+).

📉 Lossy video compression (H.264) erases those mathematical boundaries, causing severe performance collapse on real-world datasets.

🔗 WGC-Net they target the exact frequencies compression discards. suffered complete fake recall collapse — Wavelet and Laplacian priors are most fragile under compression since

⚡ Static mathematical priors are highly accurate on raw images but require adaptation strategies (augmentation, learnable transforms) to survive compression.

🔑 **Key Takeaway:** Static mathematical priors (Wavelets / Laplacians) are highly accurate on raw images but fragile against lossy video compression in the real world.

The Future of Mathematically-Driven Detection

Slide 5 · Conclusion & Future Scope

Final Conclusion

Both custom architectures successfully demonstrate that integrating established image-processing mathematics with deep learning yields highly accurate, specialized detection systems. However, our cross-dataset evaluation proves that future systems must be engineered specifically to survive video compression.

Future Scope

Vision Transformers for Global Context:

Transitioning from local CNN operations to Vision Transformer architectures. While WGC-Net excels at finding localized blending seams, ViTs use global self-attention to detect long-range inconsistencies—such as unnatural lighting variations across the face or anatomical asymmetries—that remain intact even after heavy video compression..

Cross-Generator Dataset Expansion

To bridge the generalization gap, future training pipelines must move beyond single-source datasets. By training the architecture on a highly diverse corpus covering a wide variety of manipulation techniques (e.g., GANs, Diffusion Models, FaceSwap) and real-world compression levels, we can force the model to learn universal forgery representations rather than overfitting to one specific algorithm's noise pattern

Real-Time Video

Adapt the Dual-Stream architecture to process sequential frames for real-time video deepfake detection, leveraging temporal consistency as an additional detection signal.

Sparsh Rastogi | Roll No. 2301AI56

Key References & Literature

[1] Rössler, A., et al. (2019). *"FaceForensics++: Learning to Detect Manipulated Facial Images."* ICCV. (Dataset & Compression Analysis)

[2] Li, Y., et al. (2020). *"Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics."* CVPR.

[3] Qian, Y., et al. (2020). *"Thinking in Frequency: Face Forgery Detection by Mining Frequency-aware Clues."* ECCV. (Dual-Stream Theory)

[4] Frank, J., et al. (2020). *"Leveraging Frequency Analysis for Deep Fake Image Recognition."* ICML. (Wavelet & GAN Artifact Theory)

[5] Woo, S., et al. (2018). *"CBAM: Convolutional Block Attention Module."* ECCV.

[6] Tan, M., & Le, Q. (2019). *"EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks."* ICML.

Thank
you!!